

What is RestTemplate ?

`RestTemplate` is a Spring class used to make synchronous HTTP/REST calls from one application to another.

In simple terms:

RestTemplate allows a Spring Boot application to act as an HTTP client and call another REST API.

For example, suppose you have:

```
Order Service
|
| HTTP GET
↓
Payment Service
```



The **Order Service** can use `RestTemplate` to call the **Payment Service**.

Simple example

Suppose Payment Service exposes:

```
</> http
```



Order Service can call it using:

```
RestTemplate restTemplate = new RestTemplate();

Payment payment =
    restTemplate.getForObject(
        "http://localhost:8082/payments/101",
        Payment.class
    );
```

Here:

- `RestTemplate` → HTTP client
- `getForObject()` → sends HTTP GET request
- URL → target service endpoint
- `Payment.class` → converts JSON response into a `Payment` object

If Payment Service returns:

```
{
  "id": 101,
  "status": "SUCCESS"
}
```

JSON

Spring converts that JSON into:

```
</> Java
```

Why is it called `RestTemplate` ?

Because it provides convenient methods for common REST operations:

HTTP	RestTemplate method
GET	<code>getForObject()</code>
GET	<code>getForEntity()</code>
POST	<code>postForObject()</code>
POST	<code>postForEntity()</code>
PUT	<code>put()</code>
DELETE	<code>delete()</code>
Any HTTP method	<code>exchange()</code>

Example:

</> Java

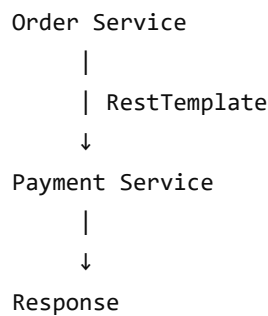


</> Java



In Microservices

A common use case is **synchronous inter-service communication**:



For example:

```
Payment payment = restTemplate.getForObject(
    "http://payment-service/payments/101",
    Payment.class
);
```

</> Java



However, there is an important issue here:

```
http://payment-service
```



may not correspond to a fixed server.

If you have:

```
Payment Service
├─ Instance 1 → 10.0.0.5:8081
├─ Instance 2 → 10.0.0.6:8081
└─ Instance 3 → 10.0.0.7:8081
```



you need **service discovery + load balancing** to determine which instance should receive the request.

This is where concepts such as **Eureka + Spring Cloud LoadBalancer** can come into the picture.

Important point for interviews

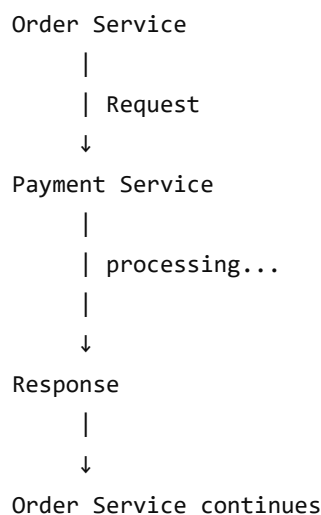
`RestTemplate` is **synchronous and blocking**.

For example:

`</>` Java



The calling thread waits until the HTTP response comes back.



Modern Spring applications may instead use:

- **WebClient** → reactive/non-blocking HTTP client
- **RestClient** → modern synchronous HTTP client in Spring Framework 6+

So, for a new Spring application, you should know `RestTemplate` because it is widely encountered in existing code, but it is **generally not the preferred choice for new development**.

One-line interview definition

`RestTemplate` is Spring's synchronous, blocking HTTP client used to communicate with REST APIs and other services over HTTP.



anything more need to learn on rest template?

Yes. For your **5.5-year Java/Spring Boot + microservices interview preparation**, you don't need to go extremely deep into `RestTemplate`. Focus on these topics:

Must know

1. Basic operations

- `getForObject()`
- `getForEntity()`
- `postForObject()`
- `postForEntity()`
- `put()`
- `delete()`
- `exchange()`

2. `getForObject()` vs `getForEntity()`

- `getForObject()` → returns response body
- `getForEntity()` → returns `ResponseEntity` containing status, headers, body

3. Path variables & query parameters

```
restTemplate.getForObject(  
    "/users/{id}",  
    User.class,  
    id  
);
```

4. Request body / response body

- Sending DTO as JSON
- Converting JSON → Java object
- `HttpMessageConverter`

5. HTTP headers

```
HttpHeaders headers = new HttpHeaders();
```

</> Java

6. `exchange()`

Very important because it allows you to control:

- HTTP method
- headers
- request body
- response type

7. Exception handling

Know:

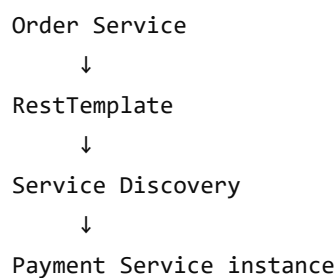
- `HttpClientErrorException`
- `HttpServerErrorException`
- `ResourceAccessException`
- `RestClientException`

8. Timeouts

Understand connection timeout and read/response timeout.

9. RestTemplate + service discovery/load balancing

Particularly relevant to your microservices preparation:



```
graph TD;
    A[Order Service] --> B[RestTemplate];
    B --> C[Service Discovery];
    C --> D[Payment Service instance];
```



10. RestTemplate + Eureka

Understand how a logical service name can be resolved to an actual service instance when Spring Cloud LoadBalancer is involved.

Good to know

- `RestTemplate` bean configuration
- Interceptors
- Error handlers (`ResponseErrorHandler`)
- Connection pooling
- Logging HTTP requests/responses
- Retry mechanisms
- Authentication with headers
- `ParameterizedTypeReference<T>` for generic responses

Example:

```
ResponseBody<List<User>> response =  
    restTemplate.exchange(  
        url,  
        HttpMethod.GET,  
        null,  
        new ParameterizedTypeReference<List<User>>() {}  
    );
```

Don't spend much time on

For your target interviews, you **don't need to memorize every** RestTemplate **method or internal implementation.**

The important understanding is:

```
RestTemplate  
    ↓  
HTTP Request  
    ↓  
Remote REST API  
    ↓  
HTTP Response  
    ↓  
JSON → Java Object
```



And know why modern applications may use RestClient or WebClient instead.

If you're covering Microservices now, the next important topic after RestTemplate is:
Inter-Service Communication → RestTemplate vs WebClient vs OpenFeign → synchronous vs asynchronous communication.



explain each in detail without missing anything

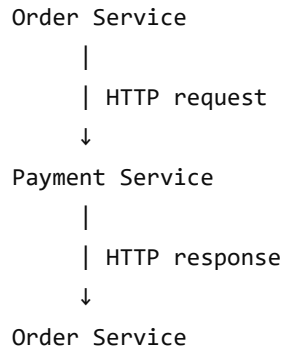
Absolutely. For your interview level, I'd learn RestTemplate in this order. I'll cover the important API usage, internals/concepts, error handling, timeouts, load balancing, Eureka, retries, interceptors, and the modern alternatives.

1. What is RestTemplate ?

`RestTemplate` is a Spring Framework class used to make **HTTP requests to external REST APIs**.

It is an **HTTP client**.

For example:



The Order Service can use `RestTemplate` to communicate with Payment Service.

Example:

```
RestTemplate restTemplate = new RestTemplate();

Payment payment = restTemplate.getForObject(
    "http://localhost:8082/payments/101",
    Payment.class
);
```

The important characteristics are:

- HTTP client
- REST API communication
- synchronous
- blocking
- request/response model
- converts Java objects ↔ JSON using message converters
- supports GET, POST, PUT, DELETE, PATCH, etc.

2. Why do we need `RestTemplate`?

Suppose another service exposes:

</> http



Without an HTTP client, you'd have to manually deal with things such as:

```
Create HTTP connection
↓
Set HTTP method
↓
Set headers
↓
Send request
↓
Receive response
↓
Read response body
↓
Convert JSON to Java object
↓
Handle HTTP errors
↓
Close connection
```



RestTemplate provides a convenient abstraction over this.

You can simply write:

```
Payment payment = restTemplate.getForObject(
    url,
    Payment.class
);
```

</> java



3. How does RestTemplate work?

Suppose:

```
Payment payment = restTemplate.getForObject(
    "http://localhost:8082/payments/101",
    Payment.class
);
```

</> java



Conceptually:

Your Java code



RestTemplate



HTTP client infrastructure



HTTP GET



Payment Service



HTTP 200 + JSON



Message Converter



Payment Java object



If the server returns:

```
{
  "id": 101,
  "status": "SUCCESS"
}
```



Jackson can convert it into:

</> Java



4. Creating a RestTemplate

For simple Java usage:

</> Java



In Spring Boot, you normally configure it as a Spring bean.

```
@Configuration
public class RestTemplateConfig {
```

```
@Bean
public RestTemplate restTemplate() {
    return new RestTemplate();
}
```

Then inject it:

```
@Service
public class PaymentClient {

    private final RestTemplate restTemplate;

    public PaymentClient(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }
}
```

Why make it a bean?

Because then Spring manages the object and you can configure things centrally:

```
RestTemplate Bean
├─ timeout
├─ interceptors
├─ error handler
├─ message converters
└─ HTTP client configuration
```

5. GET request — getForObject()

This is one of the most commonly used methods.

```
Payment payment = restTemplate.getForObject(
    "http://localhost:8082/payments/101",
    Payment.class
);
```

The arguments are essentially:

```
URL
Response type
```

The return value is the response body converted into the specified Java type.

If response is:

```
{
  "id": 101,
  "status": "SUCCESS"
}
```

</> JSON

you get:

</> Java

6. getForEntity()

Another GET method:

```
ResponseEntity<Payment> response =
    restTemplate.getForEntity(
        "http://localhost:8082/payments/101",
        Payment.class
    );
```

The difference is important.

`getForObject()`

You mainly get:

Response Body

`getForEntity()`

You get:

```
ResponseEntity
├── HTTP status
├── headers
└── body
```

Example:

```
ResponseEntity<Payment> response =  
    restTemplate.getForEntity(url, Payment.class);  
  
HttpStatusCode status = response.getStatusCode();  
  
HttpHeaders headers = response.getHeaders();  
  
Payment payment = response.getBody();
```

So:

```
getForObject()  
    ↓  
Payment  
  
getForEntity()  
    ↓  
ResponseEntity<Payment>  
    ↓  
status + headers + body
```



7. Path variables

Suppose your API is:

```
</> http
```



You can do:

```
User user = restTemplate.getForObject(  
    "http://localhost:8081/users/{id}",  
    User.class,  
    101  
);
```

Spring replaces:

`{id}`

with:

`101`

Result:

`</> http`

You can also use a map:

```
Map<String, Object> params = new HashMap<>();
params.put("id", 101);

User user = restTemplate.getForObject(
    "http://localhost:8081/users/{id}",
    User.class,
    params
);
```

8. Query parameters

Suppose API is:

`</> http`

You can construct the URL using `UriComponentsBuilder` :

```
URI uri = UriComponentsBuilder
    .fromHttpUrl("http://localhost:8081/users")
    .queryParams("name", "John")
    .queryParams("age", 28)
    .build()
    .toUri();

User[] users = restTemplate.getForObject(
    uri,
```

```
User[].class  
);
```

This is preferable to manually concatenating values because URL encoding is handled properly.

9. POST request — postForObject()

Suppose you want to create a payment.

Request:

```
</> http
```



Body:

```
{  
  "orderId": 101,  
  "amount": 5000  
},  
"application/json"
```



Java:

```
PaymentRequest request =  
    new PaymentRequest(101, 5000);  
  
PaymentResponse response =  
    restTemplate.postForObject(  
        "http://localhost:8082/payments",  
        request,  
        PaymentResponse.class  
    );
```

Here:

```
PaymentRequest  
    ↓  
JSON  
    ↓  
HTTP POST  
    ↓
```



```
Payment Service
    ↓
JSON
    ↓
PaymentResponse
```

Jackson/message converters normally handle the Java ↔ JSON conversion.

10. postForEntity()

Similar to `postForObject()`, but returns `ResponseEntity`.

```
ResponseEntity<PaymentResponse> response =
    restTemplate.postForEntity(
        url,
        request,
        PaymentResponse.class
    );
```

Then:

```
response.getStatusCode();
response.getHeaders();
```

</> Java



11. postForLocation()

Sometimes a POST operation creates a resource and the server returns a `Location` header.

Example:

```
HTTP/1.1 201 Created
```

</> http



You can use:

```
URI location =  
    restTemplate.postForLocation(  
        url,  
        request  
    );
```

You get the URI of the newly created resource.

12. PUT request

For updating an existing resource:

```
restTemplate.put(  
    "http://localhost:8082/payments/{id}",  
    request,  
    101  
);
```

`put()` doesn't return the response body.

If you need more control over the response, use `exchange()`.

13. DELETE request

```
restTemplate.delete(  
    "http://localhost:8082/payments/{id}",  
    101  
);
```

This sends:

```
</> http
```

Again, `delete()` doesn't return a response body.

14. exchange() — very important

For interviews, **understand** `exchange()` **well**.

It is a more general method.

It allows you to specify:

- HTTP method
- URL
- request entity
- request headers
- request body
- response type

Example:

```
HttpHeaders headers = new HttpHeaders();

headers.setBearerAuth(token);

HttpEntity<Void> entity =
    new HttpEntity<>(headers);

ResponseEntity<Payment> response =
    restTemplate.exchange(
        url,
        HttpMethod.GET,
        entity,
        Payment.class
    );
```

Conceptually:

```
exchange()
├──
├── URL
├── HTTP Method
├── Request Entity
│   ├── headers
│   └── body
└── Response Type
```



15. Why use `exchange()` ?

Suppose you need:

```
GET /payments/101
Authorization: Bearer abc123
```

`</> http`



You need custom headers.

`exchange()` is convenient:

```
HttpHeaders headers = new HttpHeaders();

headers.setBearerAuth(token);
headers.set("X-Correlation-ID", correlationId);

HttpEntity<Void> request =
    new HttpEntity<>(headers);

ResponseEntity<Payment> response =
    restTemplate.exchange(
        url,
        HttpMethod.GET,
        request,
        Payment.class
    );
```

16. HttpEntity

`HttpEntity` represents the HTTP request information you want to send.

It can contain:

```
HttpEntity
├── Headers
└── Body
```



Example:

```
HttpHeaders headers = new HttpHeaders();
```

```
headers.setContentType(MediaType.APPLICATION_JSON);

PaymentRequest paymentRequest =
    new PaymentRequest(101, 5000);

HttpEntity<PaymentRequest> entity =
    new HttpEntity<>(paymentRequest, headers);
```

Now:

```
HttpEntity
├─ Content-Type: application/json
└─ Body: PaymentRequest
```

17. ResponseEntity

`ResponseEntity` represents the HTTP response.

It contains:

```
ResponseEntity
├─ Status
├─ Headers
└─ Body
```

Example:

```
ResponseEntity<Payment> response = ...;

response.getStatusCode();
response.getHeaders();
response.getBody();
```

This is particularly useful when your code needs to inspect HTTP metadata.

18. HTTP headers

Headers carry metadata about the HTTP request/response.

Common headers:

Content-Type
Accept
Authorization
User-Agent
Correlation ID



Example:

```
HttpHeaders headers = new HttpHeaders();

headers.setContentType(MediaType.APPLICATION_JSON);
headers.setAccept(List.of(MediaType.APPLICATION_JSON));
headers.setBearerAuth(token);
```



Then:

```
HttpEntity<PaymentRequest> entity =
```

</> Java



19. Sending JSON

Suppose:

```
PaymentRequest request =
```

</> Java



When using a properly configured RestTemplate , Spring's message converters can convert:

Java Object



JSON



For example:

```
{
  "orderId": 101,
```

```
"amount": 5000
```

```
</> JSON
```



The response conversion works in reverse:

```
JSON
```



```
Jackson
```



```
Java Object
```



20. HttpMessageConverter

This is an important underlying concept.

RestTemplate uses HttpMessageConverter implementations to convert HTTP request/response bodies.

For JSON, a commonly used converter is based on Jackson:

```
Java Object
```



```
HttpMessageConverter
```



```
JSON
```



and:

```
JSON
```



```
HttpMessageConverter
```



```
Java Object
```



For example:

```
</> Java
```



can be populated from:

```
{  
  "id": 101.  
}</> JSON
```



You generally don't manually call Jackson for ordinary REST calls.

21. Generic responses

Suppose the server returns:

```
[  
  {  
    "id": 1,  
    "name": "John"  
  },  
  {  
    "id": 2,  
    "name": "David"  
  }  
]
```

For simple arrays:

```
User[] users =  
    restTemplate.getForObject(  
        url,  
        User[].class  
    );
```



But generic types require more care.

Suppose:

```
</> Java
```



You can use:

```

ResponseEntity<List<User>> response =
    restTemplate.exchange(
        url,
        HttpMethod.GET,
        null,
        ...
    );

```

</> Java



Why?

Java's generic type information is affected by **type erasure**, so simply passing:

</> Java



doesn't tell Spring that the list contains `User`.

`ParameterizedTypeReference` preserves the required generic type information for the response conversion.

22. Exception handling

This is very important.

Suppose the remote service returns:

</> http



or:

</> http



`RestTemplate` normally throws exceptions for HTTP error responses.

The hierarchy includes:

```

RestClientException
├── RestClientResponseException
│   ├── HttpClientErrorException
│   └── HttpServerErrorException

```



|
└─ ResourceAccessException

23. HttpClientErrorException

Usually associated with **4xx responses**.

Examples:

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found



Example:

```
try {  
    Payment payment =  
        restTemplate.getForObject(  
            url,  
            Payment.class  
        );  
} catch (HttpClientErrorException e) {  
  
    System.out.println(e.getStatusCode());  
    System.out.println(e.getResponseBodyAsString());  
}
```

24. HttpServerErrorException

Usually associated with **5xx responses**.

Examples:

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable



Example:

```
catch (HttpServerErrorException e) {  
    // Handle remote server error
```

</> Java



25. ResourceAccessException

This generally represents I/O/resource-level problems.

Examples:

Connection refused
Connection timeout
Read timeout
Network problem



Example:

```
catch (ResourceAccessException e) {  
    // Network/connection-level failure
```

</> Java



This is different from receiving an HTTP 500.

Important distinction

HTTP 500
↓
Server responded
↓
HttpServerErrorException



Whereas:

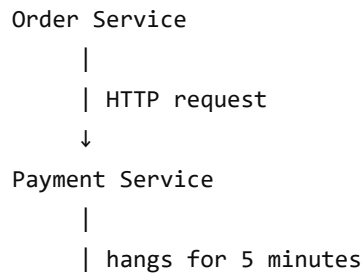
Cannot connect
↓
No HTTP response
↓
ResourceAccessException



26. Timeout

This is extremely important in microservices.

Imagine:

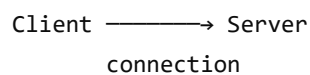


Without appropriate timeouts, the calling thread can remain blocked for too long.

Common concepts:

Connection timeout

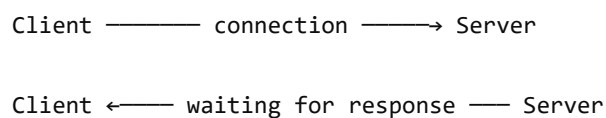
Maximum time to establish a connection.



If connection cannot be established within the configured period, fail.

Read/response timeout

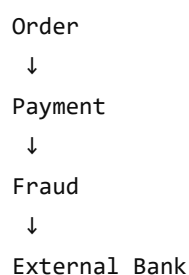
Maximum time waiting for data after the connection is established.



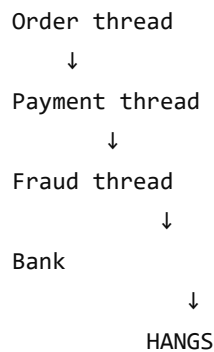
If the server doesn't respond within the configured limit, fail.

27. Why timeouts matter in microservices

Suppose:



If External Bank takes forever:



This can cause resource exhaustion and cascading failures.

Therefore, timeout configuration is essential for synchronous communication.

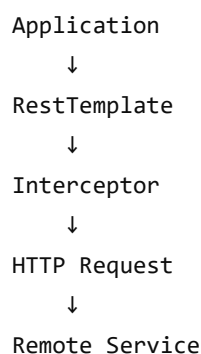
28. Interceptors

`ClientHttpRequestInterceptor` allows you to intercept outgoing HTTP requests.

Useful for:

- logging
- correlation IDs
- authentication headers
- tracing
- metrics
- common headers

Conceptually:



Example:

```
public class LoggingInterceptor
    implements ClientHttpRequestInterceptor {

    @Override
    public ClientHttpResponse intercept(
        HttpRequest request,
        byte[] body,
        ClientHttpRequestExecution execution)
        throws IOException {

        System.out.println(request.getURI());

        return execution.execute(request, body);
    }
}
```

Then configure it:

```
restTemplate.setInterceptors(
    List.of(new LoggingInterceptor())
```

</> Java



29. ClientHttpRequestInterceptor vs filter

Don't confuse it with a server-side Servlet filter.

Server filter

Runs when your application **receives** an HTTP request.

```
Client
↓
Filter
↓
Controller
```



RestTemplate interceptor

Runs when your application **sends** an HTTP request through RestTemplate.

```
Controller/Service
↓
RestTemplate
↓
Interceptor
```





Remote Service

30. ResponseErrorHandler

You can customize how HTTP errors are handled.

RestTemplate uses a ResponseErrorHandler .

You can provide your own implementation if the default behavior isn't suitable.

Conceptually:

```
HTTP Response
  ↓
ResponseErrorHandler
  ↓
Is it an error?
  ↓
Handle accordingly
```



This can be useful when you want centralized error handling for a particular RestTemplate configuration.

31. RestTemplate and authentication

Suppose Payment Service requires:

```
</> http
```



You can set it manually:

```
HttpHeaders headers = new HttpHeaders();

headers.setBearerAuth(token);

HttpEntity<Void> entity =
    new HttpEntity<>(headers);

ResponseEntity<Payment> response =
    restTemplate.exchange(
```

```
url,  
HttpMethod.GET,  
entity,  
Payment.class  
);
```

In real microservices, authentication/token propagation is often handled through an interceptor or dedicated client configuration rather than repeating the same code everywhere.

32. RestTemplate + service discovery

This is very important for your current **Microservices** preparation.

Suppose:

Payment Service

- ├ Instance 1 → 10.0.0.10:8080
- ├ Instance 2 → 10.0.0.11:8080
- └ Instance 3 → 10.0.0.12:8080



You don't necessarily want:

```
</> Java
```



because the instance can change.

Instead, conceptually:

Order Service

↓

RestTemplate

↓

Service name

↓

Service Discovery

↓

Available instances

↓

Load Balancer

↓

One Payment instance



33. RestTemplate + Eureka

With Spring Cloud, you can use a logical service name such as:

`http://PAYMENT-SERVICE/payments/101`



instead of:

`http://10.0.0.10:8080/payments/101`



Eureka maintains information such as:

PAYMENT-SERVICE

```
|
|— instance A
|— instance B
|— instance C
```



Spring Cloud LoadBalancer can select an instance.

So don't think:

"RestTemplate itself is Eureka."

They are different responsibilities.

RestTemplate

↓

HTTP client

Eureka

↓

Service registry/discovery

Spring Cloud LoadBalancer

↓

Instance selection



34. Client-side load balancing

Suppose Eureka provides:

Payment-1

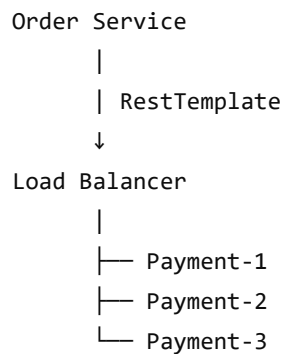
Payment-2



Payment-3

The client-side load balancer chooses an instance according to its configured strategy.

Conceptually:



This is called **client-side load balancing** because the caller participates in choosing the destination instance.

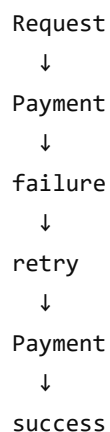
35. RestTemplate and retries

Suppose:

Order → Payment

and Payment temporarily fails.

A retry mechanism can potentially try again.



But **don't blindly retry every failure**.

For example, retrying:

```
</> http
```



could potentially create duplicate payments if the first request actually succeeded but the response was lost.

Therefore retry design depends on:

- HTTP method
- idempotency
- error type
- timeout
- business operation
- maximum attempts
- backoff

In Spring Cloud applications, retry is often implemented using dedicated resilience/retry mechanisms rather than manually adding loops around every `RestTemplate` call.

36. Idempotency and RestTemplate

This is a good interview concept.

An operation is **idempotent** if repeating it produces the same intended result.

For example:

```
</> http
```



setting the same user state repeatedly is generally intended to be idempotent.

But:

```
</> http
```



could create multiple resources if repeated.

So:

```
Timeout
↓
Should I retry?
↓
What operation?
↓
Is it safe to repeat?
```



This matters when designing retry mechanisms.

37. Connection pooling

HTTP connections can potentially be reused instead of creating a completely new connection for every request.

Without efficient connection management:

```
Request 1 → create connection → close
Request 2 → create connection → close
Request 3 → create connection → close
```



With connection pooling:

```
Connection Pool
├─ Connection 1
├─ Connection 2
├─ Connection 3
└─ ...
```



Requests can reuse connections.

This becomes important in high-throughput microservices.

The exact underlying HTTP client configuration depends on the Spring/Spring Boot version and chosen request factory.

38. Logging

In production, you often need to know:

```
Which service was called?
Which endpoint?
Which HTTP method?
Which correlation ID?
```



How long did it take?
What status came back?

An interceptor can help:

```
Request
↓
Log
↓
Remote Service
↓
Response
↓
Log
```



Be careful about logging sensitive information such as:

Authorization tokens
Passwords
Personal data
Payment information



39. Correlation ID

Suppose:

```
Client
↓
Order Service
↓
Payment Service
↓
Notification Service
```



A request may travel through multiple services.

A correlation ID can allow you to associate logs belonging to the same business request.

Example:

```
</> http
```



An interceptor can automatically add this header to outgoing requests.

Then:

```
Order logs      → abc-123
Payment logs    → abc-123
Notification logs → abc-123
```



This makes distributed troubleshooting easier.

40. Thread blocking

This is one of the most important characteristics of `RestTemplate`.

Suppose:

```
Payment payment =
```

```
</> Java
```



The calling thread waits.

```
Thread
|
| sends request
↓
Payment Service
|
| processing...
| processing...
| processing...
↓
response
|
↓
Thread continues
```



Therefore:

`RestTemplate` is synchronous and blocking.

This is different from a reactive client such as `WebClient`.

41. RestTemplate vs WebClient

You should know this comparison.

	RestTemplate	WebClient
Style	Synchronous	Reactive
Blocking	Yes	Can be non-blocking
API style	Traditional	Reactive
Common use	Existing applications	Reactive/newer applications
Return	Java object / ResponseEntity	Mono / Flux
Streaming	Limited/traditional	Strong support

Example RestTemplate:

```
Payment payment =
```

```
</> Java
```



WebClient:

```
Mono<Payment> payment =  
    webClient  
        .get()  
        .uri(url)  
        .retrieve()  
        .bodyToMono(Payment.class);
```

42. RestTemplate vs RestClient

This is another important modern Spring interview topic.

Spring Framework introduced `RestClient` as a modern synchronous HTTP client API.

So conceptually:

```
RestTemplate
```

```
↓
```

```
Older synchronous API
```

```
RestClient
```



```
↓  
Modern synchronous API  
  
WebClient  
↓  
Reactive/non-blocking API
```

Don't say:

"RestTemplate is completely removed."

That's incorrect.

It remains available, but for new development, `RestClient` is generally the modern synchronous choice.

43. RestTemplate vs OpenFeign

This is especially important in microservices interviews.

RestTemplate

You explicitly write the HTTP client call:

```
Payment payment =  
    restTemplate.getForObject(  
        url,  
        Payment.class  
    );
```

OpenFeign

You define an interface:

```
@FeignClient(name = "payment-service")  
public interface PaymentClient {  
  
    @GetMapping("/payments/{id}")  
    Payment getPayment(@PathVariable Long id);  
}
```

Then:

```
Payment payment =
```

```
</> Java
```



Conceptually:

```
RestTemplate
```



```
Service
```

```
↓
```

```
RestTemplate
```

```
↓
```

```
HTTP
```

```
Feign
```

```
Service
```

```
↓
```

```
Feign Interface
```

```
↓
```

```
HTTP
```

Feign is declarative, so it can significantly reduce boilerplate.

44. Important distinction: HTTP client vs service discovery

Don't mix these concepts.

RestTemplate

Answers:

"How do I make an HTTP request?"

Eureka

Answers:

"Where are the available instances of this service?"

LoadBalancer

Answers:

"Which available instance should I call?"

API Gateway

Answers:

"How should external/client traffic enter and be routed across services?"

Kafka

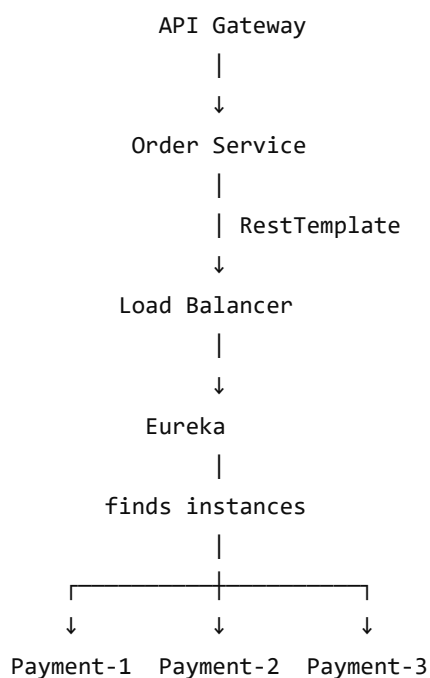
Answers:

"How can services communicate asynchronously through events/messages?"

These solve different problems.

45. Complete microservice picture

Put everything together:



More accurately, Eureka provides discovery information and the load balancer uses available service instances; the exact integration depends on the Spring Cloud setup.

46. Complete example

A reasonably realistic client:

```
@Service
public class PaymentClient {

    private final RestTemplate restTemplate;

    public PaymentClient(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    public Payment getPayment(Long id) {

        String url =
            "http://localhost:8082/payments/{id}";

        try {

            ResponseEntity<Payment> response =
                restTemplate.getForEntity(
                    url,
                    Payment.class,
                    id
                );

            return response.getBody();

        } catch (HttpClientErrorException e) {

            throw new RuntimeException(
                "Payment service returned client error",
                e
            );

        } catch (HttpServerErrorException e) {

            throw new RuntimeException(
                "Payment service returned server error",
                e
            );

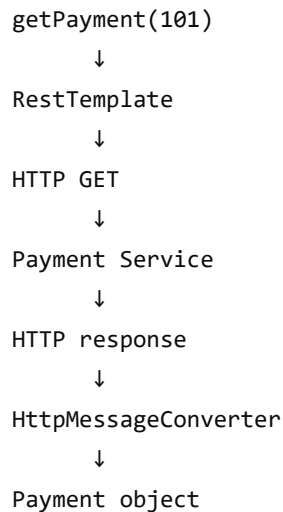
        } catch (ResourceAccessException e) {

            throw new RuntimeException(
                "Unable to connect to Payment service",
                e
            );

        }

    }
}
```

The flow is:



```
graph TD; A[getPayment(101)] --> B[RestTemplate]; B --> C[HTTP GET]; C --> D[Payment Service]; D --> E[HTTP response]; E --> F[HttpMessageConverter]; F --> G[Payment object];
```

47. What you should know for a 5–6 year interview

I'd divide the RestTemplate syllabus like this:

● Must master

1. What is RestTemplate ?
2. Synchronous vs blocking
3. getForObject()
4. getForEntity()
5. postForObject()
6. postForEntity()
7. put()
8. delete()
9. exchange()
10. HttpEntity
11. ResponseEntity
12. Request/response headers
13. Path variables
14. Query parameters
15. JSON ↔ Java conversion
16. HttpMessageConverter
17. Exception handling
18. HttpClientErrorException
19. HttpServerErrorException
20. ResourceAccessException
21. Timeouts

- 22. RestTemplate + service discovery
- 23. RestTemplate + Spring Cloud LoadBalancer
- 24. RestTemplate + Eureka

● Should know

- 25. Interceptors
- 26. ResponseErrorHandler
- 27. ParameterizedTypeReference
- 28. Authentication headers
- 29. Correlation IDs
- 30. Connection pooling
- 31. Retry concepts
- 32. Idempotency
- 33. Logging

● Know conceptually

- 34. RestTemplate vs WebClient
- 35. RestTemplate vs RestClient
- 36. RestTemplate vs OpenFeign
- 37. Why blocking calls can be problematic
- 38. Why timeouts/retries/circuit breakers matter in distributed systems

One interview question that connects almost everything

If interviewer asks:

"How does Order Service communicate with Payment Service when you have multiple Payment Service instances?"

You should be able to explain:

```
Order Service
  ↓
RestTemplate / HTTP client
  ↓
Service name
  ↓
Service Discovery (Eureka)
  ↓
Available Payment instances
  ↓
Spring Cloud LoadBalancer
  ↓
```



Selected Payment instance



HTTP response



JSON → Payment DTO

And then mention:

Because `RestTemplate` is synchronous and blocking, timeout, error handling, retry strategy, and resilience are important in production.

That is the level of understanding I'd target before moving on from `RestTemplate`.



ChatGPT can make mistakes. Check important info.